EXP 4	Implementation of self-organizing maps for a specific application.

AIM: Understand the concept of self-organizing maps for a specific application using python programming language.

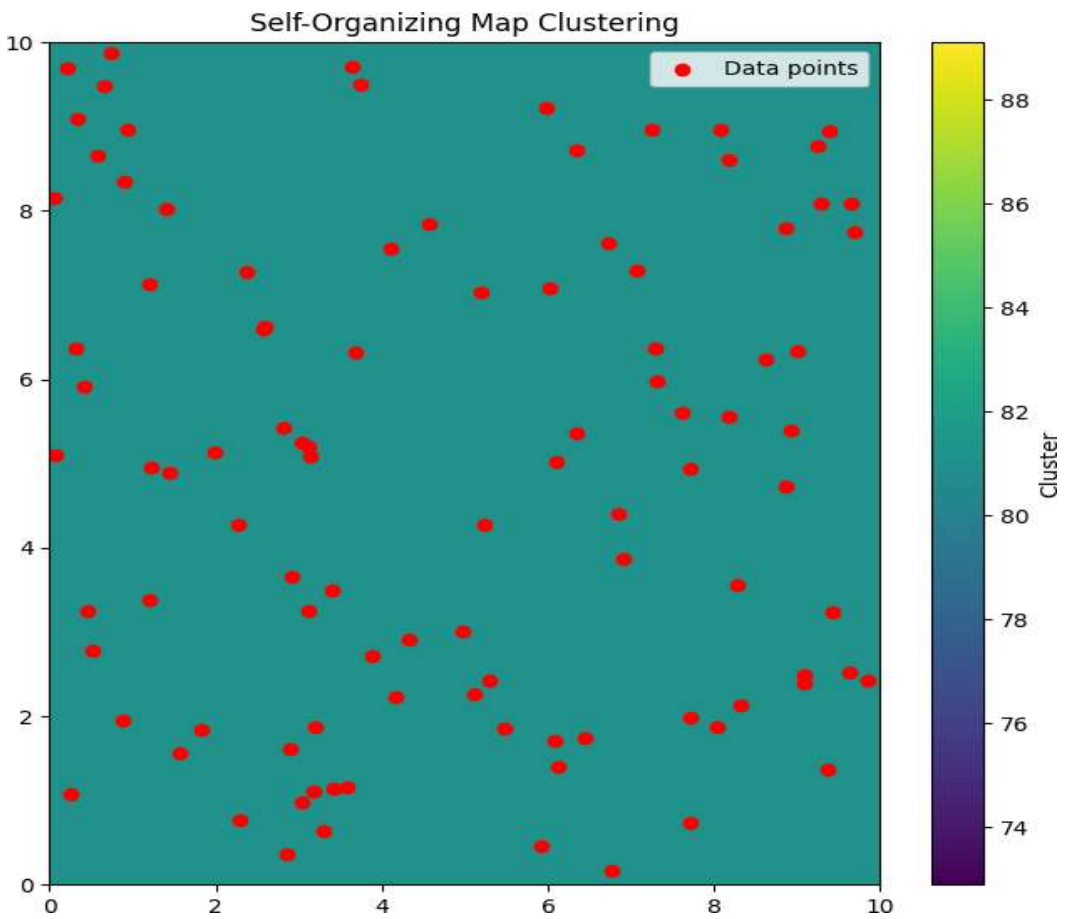
Algorithm:

1. Initialize the SOM.
   1.1 Define the grid size and topology.
   1.2 Initialize neuron weight vectors randomly.
   1.3 Set the learning rate and neighborhood radius.
2. Prepare the training dataset.
3. Train the SOM.
   3.1 For each epoch:
      3.1.1 Select an input vector randomly.
      3.1.2 Find the Best Matching Unit (BMU).
      3.1.3 Update the weights of the BMU and its neighbors.
      3.1.4 Reduce the learning rate and neighborhood radius.
4. Repeat training until convergence or maximum epochs.
5. Map new data using the trained SOM and obtain clusters.

PROGRAM:

```
import numpy as np
import matplotlib.pyplot as plt

# Generate some sample data (replace this with your own dataset)
np.random.seed(42)
data = np.random.rand(100, 2)

# SOM parameters
grid_size = (10, 10)  # Grid size of the SOM
input_dim = 2  # Dimensionality of the input data
learning_rate = 0.2
num_epochs = 1000

# Initialize the SOM
weight_matrix = np.random.rand(grid_size[0], grid_size[1], input_dim)

# Training loop
for epoch in range(num_epochs):
    for input_vector in data:
        # Find the Best Matching Unit (BMU)
        distances = np.linalg.norm(weight_matrix - input_vector, axis=-1)
        bmu_coords = np.unravel_index(np.argmin(distances), distances.shape)

        # Update the BMU and its neighbors
        for i in range(grid_size[0]):
            for j in range(grid_size[1]):
                distance_to_bmu = np.linalg.norm(np.array([i, j]) - np.array(bmu_coords))
```

```
            influence = np.exp(-distance_to_bmu**2 / (2 * (epoch + 1)**2))
            weight_matrix[i, j] += influence * learning_rate * (input_vector - weight_matrix[i, j])

# Create a map of cluster assignments
cluster_map = np.zeros((grid_size[0], grid_size[1]), dtype=int)
for i in range(grid_size[0]):
    for j in range(grid_size[1]):
        distances = np.linalg.norm(data - weight_matrix[i, j], axis=-1)
        cluster_map[i, j] = np.argmin(distances)

# Visualize the results
plt.figure(figsize=(8, 8))
plt.pcolormesh(cluster_map, cmap='viridis')
plt.colorbar(label='Cluster')
plt.scatter(data[:, 0] * grid_size[0], data[:, 1] * grid_size[1], color='red', label='Data points')
plt.legend()
plt.title('Self-Organizing Map Clustering')
plt.show()
```



Result: Thus the above program for self-organizing map executed successfully with desired output.